

Linear equations : iterative methods

Gauss-Jordan and LU quickly become unmanageable $\rightarrow$ pivoting costly
increasing matrix size + increasing number of 0 entries (sparse matrix)

Best strategy is *iterative method* : Jacobi & Gauss-Seidel

• **Jacobi method** : works when **A** has no zeros in the main diagonal. It decomposes, but not factorises, **A** into a diagonal **D** and **L, U**

$$\mathbf{A} = \mathbf{D} + (\mathbf{L} + \mathbf{U}) = \begin{pmatrix} a_{00} & 0 & \cdots & 0 \\ 0 & a_{11} & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & a_{n-1\ n-1} \end{pmatrix} + \begin{pmatrix} 0 & a_{01} & \cdots & a_{0\ n-1} \\ a_{10} & 0 & \cdots & a_{1\ n-1} \\ \vdots & \vdots & \ddots & \vdots \\ a_{n-1\ 0} & a_{n-1\ 1} & \cdots & 0 \end{pmatrix}$$

Then the solution of $\mathbf{A} \cdot \mathbf{x}$ can be approached as

$$[\mathbf{D} + (\mathbf{L} + \mathbf{U})] \cdot \mathbf{x} = \mathbf{b} \Rightarrow \mathbf{D} \cdot \mathbf{x} = \mathbf{b} - (\mathbf{L} + \mathbf{U}) \cdot \mathbf{x} \Rightarrow \mathbf{x} = \mathbf{D}^{-1} [\mathbf{b} - (\mathbf{L} + \mathbf{U}) \cdot \mathbf{x}]$$

Above expression evaluated iteratively for $\mathbf{x}$ until $\|\mathbf{x}^{k+1} - \mathbf{x}^k\| < \epsilon$,

$$\mathbf{x}^{(k+1)} = \mathbf{D}^{-1} (\mathbf{b} - (\mathbf{L} + \mathbf{U})\mathbf{x}^{(k)}) \Rightarrow x_i^{(k+1)} = \frac{1}{a_{ii}} \left(b_i - \sum_{j \neq i} a_{ij} x_j^{(k)} \right)$$

Jacobi method is fairly fast but computation of $x_i^{(k+1)}$ element requires all other elements, we cannot overwrite $x_i^{(k)}$ with $x_i^{(k+1)}$.

Sufficient condition for the method to converge is that the matrix A is **strictly diagonally dominant**,

$$|a_{ii}| \gtrsim \sum_{j \neq i} |a_{ij}|$$

Although Jacobi method is relatively slower to converge to solutions, its main advantage is that it can be easily **parallelized**.

Hence it can handle large matrices, which reduces both memory requirement for matrix storage and time taken.

As an example solve the following,

$$\begin{array}{rcl} 4x_1 - x_2 + x_3 & = & 7 \\ -2x_1 + x_2 + 5x_3 & = & 15 \\ 4x_1 - 8x_2 + x_3 & = & -21 \end{array} \Rightarrow \mathbf{A} = \begin{pmatrix} 4 & -1 & 1 \\ -2 & 1 & 5 \\ 4 & -8 & 1 \end{pmatrix} \rightarrow \begin{pmatrix} 4 & -1 & 1 \\ 4 & -8 & 1 \\ -2 & 1 & 5 \end{pmatrix}$$

Above $\mathbf{A}$ is not **diagonally dominant** but swapping **eqn. 2** with **eqn. 3** i.e. **row 2** with **row 3** of $\mathbf{A}$ will make it so.

According to the formula above

$$x_1 = \frac{7+x_2-x_3}{4} \quad x_1^{(k+1)} = \frac{7+x_2^{(k)}-x_3^{(k)}}{4}$$

$$x_2 = \frac{21+4x_1+x_3}{8} \Rightarrow x_2^{(k+1)} = \frac{21-4x_1^{(k)}-x_3^{(k)}}{8}$$

$$x_3 = \frac{15+2x_1-x_2}{5} \quad x_3^{(k+1)} = \frac{15+2x_1^{(k)}-x_2^{(k)}}{5}$$

Start with initial guess $(x_1^{(0)}, x_2^{(0)}, x_3^{(0)}) = (0, 0, 0)$.

(k)	$x_1^{(k)}$	$x_2^{(k)}$	$x_3^{(k)}$
0	0	0	0
1	1.75	2.625	3.0
2	1.656	3.875	3.175
$\vdots$	$\vdots$	$\vdots$	$\vdots$
6	1.9995	3.996	2.9987

Try the following without making diagonally dominant

$$\begin{aligned} 2x + 3y + z &= 13 \\ x + y + z &= 6 \\ 3x + 2y + 2z &= 15 \end{aligned} \quad x = 3, y = 2, z = 1$$

Gauss-Seidel method

Gauss-Seidel method is defined by the relation

$$\mathbf{A} = \mathbf{L}_* + \mathbf{U} \rightarrow \begin{pmatrix} a_{00} & 0 & \cdots & 0 \\ a_{10} & a_{11} & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ a_{n-1,1} & a_{n-1,2} & \cdots & a_{n-1,n-1} \end{pmatrix} + \begin{pmatrix} 0 & a_{01} & \cdots & a_{0,n-1} \\ 0 & 0 & \cdots & a_{2,n-1} \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & 0 \end{pmatrix}$$

The solution is obtained through the iterative process $\mathbf{A} \cdot \mathbf{x} = \mathbf{b}$

$$(\mathbf{L}_* + \mathbf{U}) \cdot \mathbf{x} = \mathbf{b} \Rightarrow \mathbf{L}_* \mathbf{x}^{(k+1)} = \mathbf{b} - \mathbf{U} \mathbf{x}^{(k)}$$

Iterative process continues until the changes

$$\|\mathbf{x}^{(k+1)} - \mathbf{x}^{(k)}\| < \epsilon$$

$$x_i^{(k+1)} = \frac{1}{a_{ii}} \left(b_i - \sum_{j=0}^{i-1} a_{ij} x_j^{(k+1)} - \sum_{j=i+1}^{n-1} a_{ij} x_j^{(k)} \right), \quad i = 0, 1, \dots, n-1$$

Gauss-Seidel requires only one storage vector for $\mathbf{x}^{(k)}$ as against two in Jacobi, since computation of $\mathbf{x}^{(k+1)}$ uses elements already calculated $\mathbf{x}^{(k+1)}$ and those $\mathbf{x}^{(k)}$ that are already used in the previous iteration.

Convergence is only guaranteed if the matrix is either *strictly diagonally dominant* or *symmetric and positive definite*.

Let us try to understand **Gauss-Seidel method** through an explicit calculation in a 4×4 matrix

$$\text{RHS} = \begin{pmatrix} b_0 \\ b_1 \\ b_2 \\ b_3 \end{pmatrix} - \begin{pmatrix} 0 & a_{01} & a_{02} & a_{03} \\ 0 & 0 & a_{12} & a_{13} \\ 0 & 0 & 0 & a_{23} \\ 0 & 0 & 0 & 0 \end{pmatrix} \begin{pmatrix} x_0^{(k)} \\ x_1^{(k)} \\ x_2^{(k)} \\ x_3^{(k)} \end{pmatrix}$$

$$= \begin{pmatrix} b_0 - a_{01}x_1^{(k)} - a_{02}x_2^{(k)} - a_{03}x_3^{(k)} \\ b_1 - a_{12}x_2^{(k)} - a_{13}x_3^{(k)} \\ b_2 - a_{23}x_3^{(k)} \\ b_3 \end{pmatrix} = b_i - \sum_{j=i+1}^{n-1} a_{ij}x_j^{(k)}$$

$$\text{LHS} = \begin{pmatrix} a_{00} & 0 & 0 & 0 \\ a_{10} & a_{11} & 0 & 0 \\ a_{20} & a_{21} & a_{22} & 0 \\ a_{30} & a_{31} & a_{32} & a_{33} \end{pmatrix} \begin{pmatrix} x_0^{(k+1)} \\ x_1^{(k+1)} \\ x_2^{(k+1)} \\ x_3^{(k+1)} \end{pmatrix}$$

$$= \begin{pmatrix} a_{00}x_0^{(k+1)} \\ a_{10}x_0^{(k+1)} + a_{11}x_1^{(k+1)} \\ a_{20}x_0^{(k+1)} + a_{21}x_1^{(k+1)} + a_{22}x_2^{(k+1)} \\ a_{30}x_0^{(k+1)} + a_{31}x_1^{(k+1)} + a_{32}x_2^{(k+1)} + a_{33}x_3^{(k+1)} \end{pmatrix} = a_{ii}x_i^{(k+1)} + \sum_{j=0}^{i-1} a_{ij}x_j^{(k+1)}$$

Equating LHS with RHS

$$\begin{aligned}a_{00}x_0^{(k+1)} &= b_0 - a_{01}x_1^{(k)} - a_{02}x_2^{(k)} - a_{03}x_3^{(k)} \\a_{10}x_0^{(k+1)} + a_{11}x_1^{(k+1)} &= b_1 - a_{12}x_2^{(k)} - a_{13}x_3^{(k)} \\a_{20}x_0^{(k+1)} + a_{21}x_1^{(k+1)} + a_{22}x_2^{(k+1)} &= b_2 - a_{23}x_3^{(k)} \\a_{30}x_0^{(k+1)} + a_{31}x_1^{(k+1)} + a_{32}x_2^{(k+1)} + a_{33}x_3^{(k+1)} &= b_3 \\a_{ii}x_i^{(k+1)} + \sum_{j=0}^{i-1} a_{ij}x_j^{(k+1)} &= b_i - \sum_{j=i+1}^{n-1} a_{ij}x_j^{(k)}\end{aligned}$$

Thus we see, $x_i^{(k)}$ is used only once and can be replaced with $x_i^{(k+1)}$.
More explicitly

$$\begin{aligned}x_0^{(k+1)} &= \frac{1}{a_{00}} \left(b_0 - a_{01}x_1^{(k)} - a_{02}x_2^{(k)} - a_{03}x_3^{(k)} \right) \\x_1^{(k+1)} &= \frac{1}{a_{11}} \left(b_1 - a_{10}x_0^{(k+1)} - a_{12}x_2^{(k)} - a_{13}x_3^{(k)} \right) \\x_2^{(k+1)} &= \frac{1}{a_{22}} \left(b_2 - a_{20}x_0^{(k+1)} - a_{21}x_1^{(k+1)} - a_{23}x_3^{(k)} \right) \\x_3^{(k+1)} &= \frac{1}{a_{33}} \left(b_3 - a_{30}x_0^{(k+1)} - a_{31}x_1^{(k+1)} - a_{32}x_2^{(k+1)} \right) \\i.e. \quad x_i^{(k+1)} &= \frac{1}{a_{ii}} \left(b_i - \sum_{j=0}^{i-1} a_{ij}x_j^{(k+1)} - \sum_{j=i+1}^{n-1} a_{ij}x_j^{(k)} \right)\end{aligned}$$

Successive Over-relaxation method

Accelerates *i.e.* improves convergence of Gauss-Seidel

It introduces a weight factor ω to extrapolate the next approximate step

⇒ weighted average of $x_i^{(k)}$ and newly computed Gauss-Seidel $x_i^{(k+1)}$

$$x_i^{(k+1)} = (1 - \omega) x_i^{(k)} + \frac{\omega}{a_{ii}} \left(b_i - \sum_{j=0}^{i-1} a_{ij} x_j^{(k+1)} - \sum_{j=i+1}^{n-1} a_{ij} x_j^{(k)} \right)$$

- ▶ $\omega = 1$: Normal Gauss-Seidel
- ▶ $1 < \omega < 2$: Over-relaxation for slow convergence
- ▶ $0 < \omega < 1$: Under-relaxation for oscillating or diverging solution

An example :

$$\begin{pmatrix} 2 & -1 & 0 & \cdots & 0 \\ -1 & 2 & -1 & \cdots & 0 \\ 0 & -1 & 2 & \cdots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \cdots & 2 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \\ \vdots \\ x_n \end{pmatrix} = \begin{pmatrix} 1 \\ 1 \\ 1 \\ \vdots \\ 1 \end{pmatrix}$$

Gauss-Seidel will take about 150 iterations, while SOR will take about 25 iterations with $\omega = 1.57$ for $n \geq 3$. [Ref. ChatGPT]

Solve the following,

$$4x_1 + x_2 - x_3 + x_4 = -2$$

$$x_1 + 4x_2 - x_3 - x_4 = -1$$

$$-x_1 - x_2 + 5x_3 + x_4 = 0$$

$$x_1 - x_2 + x_3 + 3x_4 = 1$$